



Smart Contract Security Audit Report



Table Of Contents

1 Executive Summary	_____
2 Audit Methodology	_____
3 Project Overview	_____
3.1 Project Introduction	_____
3.2 Vulnerability Information	_____
4 Code Overview	_____
4.1 Contracts Description	_____
4.2 Visibility Description	_____
4.3 Vulnerability Summary	_____
5 Audit Result	_____
6 Statement	_____

1 Executive Summary

On 2025.09.28, the SlowMist security team received the Brevis Network team's security audit application for Brevis Token Vesting, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

This is a token vesting project that supports setting token allocation quotas for multiple beneficiaries and releasing them linearly according to a preset schedule. The project uses role-based permission control and supports a pause mechanism and emergency fund rescue functions.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Missing Input Validation in setAllocations Function	Design Logic Audit	Low	Partial Fixed
N2	Insufficient Balance Check in Release Function	Others	Suggestion	Acknowledged
N3	Insufficient Validation in setVestingParameters	Others	Suggestion	Acknowledged
N4	Potential Integer Overflow in Vesting Calculation	Integer Overflow and Underflow Vulnerability	Suggestion	Acknowledged
N5	Inefficient Function Visibility (public instead of external)	Others	Suggestion	Acknowledged
N6	Missing zero address check	Others	Suggestion	Partial Fixed
N7	Token Compatibility Reminder	Others	Information	Acknowledged
N8	Owner Can Withdraw All Vesting Tokens in Emergency	Others	Information	Acknowledged

4 Code Overview

4.1 Contracts Description

Audit Version:

<https://github.com/brevis-network/token-vesting>

commit: b0446032dab08c4e08a76a51aff626a2e0338b29

Fixed Version:

<https://github.com/brevis-network/token-vesting>

commit: 6b231d2a0da0da3dba439cfd175dbb992540d27b

```
./src
├── TokenAllocation.sol
└── TokenVesting.sol
```

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

TokenAllocation			
Function Name	Visibility	Mutability	Modifiers
setAllocations	External	Can Modify State	whenNotPaused onlyRole
lockAllocations	External	Can Modify State	-

TokenVesting			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
release	External	Can Modify State	whenNotPaused
release	External	Can Modify State	whenNotPaused onlyRole
_release	Internal	Can Modify State	nonReentrant
releasable	Public	-	-
vestingSchedule	Public	-	-
vestingSchedule	Public	-	-
vestingSchedule	Public	-	-
beneficiaryVestingInfo	External	-	-
fundingGap	Public	-	-

TokenVesting			
setVestingParameters	External	Can Modify State	onlyOwner
setBeneficiaryPaused	External	Can Modify State	onlyRole
setToken	External	Can Modify State	onlyOwner
recoverERC20	External	Can Modify State	onlyOwner
sweepTokens	External	Can Modify State	onlyOwner whenPaused

4.3 Vulnerability Summary

[N1] [Low] Missing Input Validation in setAllocations Function

Category: Design Logic Audit

Content

The setAllocations function in TokenAllocation contract lacks critical input validation that could lead to operational issues and gas limit denial-of-service. First, the function does not validate beneficiary addresses for zero addresses and does not implement reasonable array length limits. Second, the function's logic allows setting allocations without ensuring the contract holds a corresponding amount of the vesting token. This creates an operational risk where beneficiaries are allocated tokens that the contract cannot deliver, leading to failed release transactions until the contract is manually funded. Lastly, the function iterates over unbounded arrays, making it susceptible to a block gas limit DoS if called with a very large number of beneficiaries.

Code location:

src/TokenAllocation.sol#L29-56

```
function setAllocations(address[] calldata _beneficiaries, uint256[] calldata
_allocation)
    external
    whenNotPaused
    onlyRole(UPDATER_ROLE)
{
    require(!allocationLocked, "Allocations are locked");

    uint256 numBeneficiaries = _beneficiaries.length;
    require(numBeneficiaries == _allocations.length, "Mismatched input lengths");
```



```

uint256 currentTotalAllocation = totalAllocation;
unchecked {
    for (uint256 i = 0; i < numBeneficiaries; ++i) {
        address beneficiary = _beneficiaries[i];
        uint256 newAllocation = _allocations[i];
        uint256 currentAllocation = allocations[beneficiary];
        if (newAllocation != currentAllocation) {
            allocations[beneficiary] = newAllocation;
            // Safe arithmetic: update cached total
            currentTotalAllocation = currentTotalAllocation + newAllocation -
currentAllocation;
            emit AllocationSet(beneficiary, newAllocation);
        }
    }
}
// Single storage write at the end
totalAllocation = currentTotalAllocation;
emit TotalAllocationSet(totalAllocation);
}

```

Solution

It's recommended to implement proper validation within the setAllocations function.

1. Add 0 address require check.
2. Consider adding a maximum length check for the input arrays.
3. Ensure the contract is funded with the total allocation amount before beneficiaries can claim their tokens.

Status

Partial Fixed; The zero-address allocation fixed, the batch size cap is not added.

[N2] [Suggestion] Insufficient Balance Check in Release Function

Category: Others

Content

The _release function calculates the amount a beneficiary is entitled to and attempts a safeTransfer. However, it does not first verify if the contract's own token balance is sufficient to cover this amount. If the contract is underfunded, the safeTransfer call will revert due to the underlying ERC20 transfer failure.

Code location:

src/TokenVesting.sol#L86-L97

```
function _release(address _beneficiary) internal nonReentrant {
    ...
    token.safeTransfer(_beneficiary, amount);
    emit TokensReleased(_beneficiary, amount);
}
```

Solution

It is recommended to add a check at the beginning of the _release function to ensure the contract has a sufficient balance before proceeding with state changes and the transfer.

Status

Acknowledged

[N3] [Suggestion] Insufficient Validation in setVestingParameters

Category: Others

Content

The setVestingParameters function does not prevent setting a _vestingStartTime that is in the past (i.e., less than block.timestamp). This could cause a large, unintentional amount of tokens to be immediately claimable upon setting the parameters. It also allows _initVestedBps to be set to 10000 (100%). This creates a potentially confusing state where all tokens are unlocked at the start time, defeating the purpose of a vesting schedule and could lead to immediate token drainage.

Code location:

src/TokenVesting.sol#L200-L213

```
function setVestingParameters(uint256 _initVestedBps, uint256 _vestingStartTime,
uint256 _vestingDuration)
    external
    onlyOwner
{
    require(!allocationLocked, "Allocations locked");
    require(_initVestedBps <= BPS_DENOMINATOR, "Initial vested BPS exceeds 100%");
    require(_vestingStartTime > 0, "Vesting start time must be greater than
zero");
    require(_vestingDuration > 0, "Vesting duration must be greater than zero");

    initVestedBps = _initVestedBps;
    vestingStartTime = _vestingStartTime;
```

```

        vestingDuration = _vestingDuration;
        emit VestingParametersSet(_initVestedBps, _vestingStartTime,
        _vestingDuration);
    }

```

Solution

It's recommended to add validation to ensure `_vestingStartTime >= block.timestamp` (future time only) and consider adding a maximum limit for `_initVestedBps` (e.g., 50%) to maintain meaningful vesting schedules.

Status

Acknowledged

[N4] [Suggestion] Potential Integer Overflow in Vesting Calculation

Category: Integer Overflow and Underflow Vulnerability

Content

In the `vestingSchedule` function, the linear vesting portion is calculated via `(remaining * elapsed) / vestingDuration`.

The intermediate multiplication `remaining * elapsed` can lead to an integer overflow if the token has a very large supply (or many decimals) and the vesting duration is long.

Code location:

`src/TokenVesting.sol#L159`

```

    function vestingSchedule(uint256 _totalAmount, uint256 _timestamp) public view
returns (uint256 vestedAmount) {
    require(vestingStartTime > 0 && vestingDuration > 0, "Vesting parameters not
set");

    if (_timestamp < vestingStartTime) {
        // Before the vesting period begins, no tokens are available
        return 0;
    } else if (_timestamp >= vestingStartTime + vestingDuration) {
        // After the vesting period ends, all tokens are fully vested
        return _totalAmount;
    } else {
        ...
        uint256 linearVested = (remaining * elapsed) / vestingDuration;
        ...
    }
}

```

Solution

It is recommended to use OpenZeppelin Math.sol library's mulDiv function to prevent overflow.

Status

Acknowledged

[N5] [Suggestion] Inefficient Function Visibility (public instead of external)

Category: Others

Content

The fundingGap function is declared with public visibility but is not called internally by any other function in the contract. When a function is only meant to be called from outside the contract, external visibility is more gas-efficient than public, because for external calls, the arguments are read directly from calldata instead of being copied to memory.

Code location:

src/TokenVesting.sol#L188-L192

```
function fundingGap() public view returns (int256) {
    uint256 totalReleasable = vestingSchedule(totalAllocation, block.timestamp) -
totalReleased;
    uint256 balance = address(token) == address(0) ? 0 :
token.balanceOf(address(this));
    return int256(balance) - int256(totalReleasable);
}
```

Solution

It is recommended to change the visibility of the fundingGap function from public to external to optimize gas usage for external callers.

Status

Acknowledged

[N6] [Suggestion] Missing zero address check

Category: Others

Content

The constructor in TokenVesting contract accepts zero addresses as valid parameters for critical roles (_updater, _pauser) and token address without validation.

Code location:

TokenVesting.sol#L59-L63

```
constructor(IERC20 _token, address _updater, address _pauser) {  
    token = _token;  
    _grantRole(UPDATER_ROLE, _updater);  
    _grantRole(PAUSER_ROLE, _pauser);  
}
```

Solution

It is recommended to add the 0 address check when sensitive parameters are modified.

Status

Partial Fixed

[N7] [Information] Token Compatibility Reminder

Category: Others

Content

The contract's logic relies on standard ERC20 token behaviors and may not be compatible with certain non-standard token types. The accounting mechanism, which relies on the amount specified in a transfer with the exact amount, can lead to incorrect behavior when interacting with these tokens, e.g, Fee-on-Transfer (FoT) Tokens and Rebasing Tokens.

Solution

It is recommended not to support Fee-on-Transfer or Rebasing tokens.

Status

Acknowledged; The contract header explicitly states it's for a "fully trusted, standard ERC20 token (no fees, no rebasing).

[N8] [Information] Owner Can Withdraw All Vesting Tokens in Emergency

Category: Others

Content

The sweepTokens function grants the owner the ability to withdraw the entire balance of the vesting token from the contract. This functionality is designed as an emergency escape hatch, and its use is restricted by two conditions: it can only be called by the owner, and only when the contract is in a paused state.

Code location:

src/TokenVesting.sol#L265-L272

```
function sweepTokens(address _to, uint256 _amount) external onlyOwner whenPaused
{
    require(_to != address(0), "invalid recipient");
    require(_amount > 0, "amount must be greater than 0");

    require(_amount <= token.balanceOf(address(this)), "insufficient balance");
    token.safeTransfer(_to, _amount);
    emit TokensSwept(_to, _amount);
}
```

Solution

N/A

Status

Acknowledged; It is the project team's expected design.

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002509290001	SlowMist Security Team	2025.09.28 - 2025.09.29	Passed

Summary conclusion: The SlowMist security team uses a manual and the SlowMist team's analysis tool to audit the project. During the audit work, we found 1 low risk, 5 suggestions, and 1 information. And 1 low risk and 1 suggestion were partially fixed; others were acknowledged. The code was not deployed to the mainnet

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>